DIY Challenge Robot
Calibration Integration Guide

Master Map of All CALIB: Markers • IMU Noise Parameters
Lidar-IMU Extrinsics • Camera-Lidar Extrinsics • GPS Lever Arm
Where to Put Results & Validation Checklist

ROS 2 Humble • All config files cross-referenced

Table of Contents

1 Master Table of All CALIB Placeholders

Every CALIB: comment in the repo represents a value that **must be replaced** with a measured or calibrated result before autonomous operation. The table below is the single reference for all placeholder locations.

WARNING: Running FAST-LIO2 or LIO-SAM with placeholder extrinsic values (identity matrix) will produce severely degraded localization. Replace all CALIB placeholders before the first mapping session.

File	Parameter	Current (Placeholder)	Source of Correct Value
fast_lio_hesai_qt64.yaml mapping.acc_cov	acc_cov: 0.1	0.1 (placeholder)	imu_utils Allen-variance output (N_a^2)
fast_lio_hesai_qt64.yaml mapping.gyr_cov	gyr_cov: 0.1	0.1 (placeholder)	imu_utils Allen-variance output (N_g^2)
fast_lio_hesai_qt64.yaml mapping.b_acc_cov	b_acc_cov: 0.0001	0.0001 (placeholder)	imu_utils bias instability (B_a^2)
fast_lio_hesai_qt64.yaml mapping.b_gyr_cov	b_gyr_cov: 0.0001	0.0001 (placeholder)	imu_utils bias instability (B_g^2)
fast_lio_hesai_qt64.yaml mapping.extrinsic_T	extrinsic_T: [0,0,0]	[0,0,0] (identity)	lidar_imu_calib translation output (metres)
fast_lio_hesai_qt64.yaml mapping.extrinsic_R	extrinsic_R: [1,0,0, 0,1,0, 0,0,1]	Identity matrix	lidar_imu_calib rotation matrix output (row-major)
robot.urdf.xacro base_to_lidar origin	xyz="0.0 0.0 0.38" rpy="0 0 0"	Physical mount approx	lidar_imu_calib result + tape measure
robot.urdf.xacro base_to_imu origin	xyz="0.0 0.0 0.25" rpy="0 0 0"	Physical mount approx	Tape measure from base_link origin
robot.urdf.xacro base_to_camera origin	xyz="0.20 0.0 0.20" rpy="0 0 0"	PLACEHOLDER — tape measure only	calibrate_cam_lidar.sh extrinsic result
robot.urdf.xacro base_to_gps origin	xyz="0.0 0.0 0.42" rpy="0 0 0"	Physical mount approx	Tape measure from base_link to antenna phase centre
navsat_transform.yaml magnetic_declination_radians	0.0	0.0 (will cause heading error)	NOAA magnetic declination for competition site
navsat_transform.yaml yaw_offset	1.5708 rad (π/2)	Assumes robot faces East	Measured: angle from robot +X to magnetic North (CW)
ekf_local.yaml process_noise_covariance	Diagonal 0.05–0.06	Tunable starting point	AprilTag calibration sessions (diy-sad.html §6.3)

NOTE: The highlighted rows (amber background) are CALIB placeholders. After calibrating, delete the CALIB: comment from the YAML/URDF and commit the updated values.

2 IMU Noise Calibration → fast_lio_hesai_qt64.yaml

IMU noise parameters describe the spectral density of white noise (N_a, N_g) and the random-walk bias instability (B_a, B_g) of the accelerometer and gyroscope. They are measured from a 2-hour stationary dataset using Allan variance analysis via the imu_utils package.

2.1 Collect the IMU Dataset

Record a 2-hour stationary IMU bag:

```
# Place robot on a vibration-free surface (foam pad)
# Must be stationary for the full recording
source scripts/env.sh jetson
ros2 bag record -o ~/imu_calib_bag /imu/data
# Record for at least 2 hours (7200 seconds)
# Stop with Ctrl+C
```

NOTE: QT64 lidar spinning creates vibration that affects the IMU. Turn off the lidar (disconnect 12V rail) during the IMU calibration recording.

2.2 Run imu_utils Allan Variance

Run calibration (imu_utils must be built in third_party_ws):

```
source third_party_ws/install/setup.bash
bash scripts/calibrate_imu.sh ~/imu_calib_bag
# Runs imu_utils on the bag; outputs imu_param.yaml
# Output file location: ~/imu_utils_output/imu_param.yaml
```

2.3 Read the Output Values

Example imu_utils output (imu_param.yaml):

```
Gyr:
  unit: "rad/s"
  avg-axis:
    gyr_n:  0.0034    # ← this is N_g (noise density rad/s/√Hz)
    gyr_w:  0.000085  # ← this is B_g (bias instability rad/s/√Hz)
Acc:
  unit: "m/s2"
  avg-axis:
    acc_n:  0.018     # ← this is N_a (noise density m/s²/√Hz)
    acc_w:  0.00031   # ← this is B_a (bias instability m/s²/√Hz)
```

2.4 Write Values into fast_lio_hesai_qt64.yaml

src/diy_localization/config/fast_lio_hesai_qt64.yaml — mapping section:

```
# Replace these placeholder values with imu_utils output:
acc_cov:   0.018      # <- acc_n^2 from imu_param.yaml Acc.avg-axis.acc_n
gyr_cov:   0.0034     # <- gyr_n^2 from imu_param.yaml Gyr.avg-axis.gyr_n
b_acc_cov: 0.00031    # <- acc_w from imu_param.yaml Acc.avg-axis.acc_w
b_gyr_cov: 0.000085   # <- gyr_w from imu_param.yaml Gyr.avg-axis.gyr_w
```

WARNING: The values above are examples. Always use the actual numbers from your imu_param.yaml. Do not share IMU calibration values between different IMU units — even same-model units have unit-to-unit variation.

3 Lidar-IMU Extrinsic → fast_lio_hesai_qt64.yaml + robot.urdf.xacro

The lidar-IMU extrinsic is the rigid-body transform from the IMU frame to the lidar frame. FAST-LIO2 uses it to motion-compensate each lidar scan. It is measured using lidar_imu_calib (target-free, figure-8 motion).

3.1 Run the Extrinsic Calibration

Execute calibrate_extrinsics.sh:

```
source scripts/env.sh jetson
bash scripts/calibrate_extrinsics.sh
# Starts lidar_imu_calib
# Drives a figure-8 manoeuvre while recording
# Outputs: extrinsic_T (3-vector, metres)
#          extrinsic_R (3x3 rotation matrix, row-major)
```

- Drive a smooth figure-8 at moderate speed (~0.3 m/s) in a 4 m × 4 m space.
- The figure-8 must excite all 6 degrees of freedom for good observability.
- Run the calibration at least 3 times; keep the result with lowest residual.
- Target residual: < 0.5 cm translation, < 0.5° rotation.

3.2 Write Extrinsics into FAST-LIO2 Config

src/diy_localization/config/fast_lio_hesai_qt64.yaml — mapping section:

```
# Replace placeholder identity with lidar_imu_calib output:
extrinsic_T: [ 0.012, -0.003, 0.131 ]   # <- T from calibration (example)
extrinsic_R: [ 0.9998,  0.0021, -0.0198,  # <- R row-major (example)
               -0.0018, 0.9999,  0.0134,
                0.0198, -0.0134, 0.9997 ]
extrinsic_est_en: false   # disable online estimation after calibration
```

3.3 Write Extrinsics into URDF

src/diy_robot_description/urdf/robot.urdf.xacro — base_to_lidar joint:

```
<!-- Replace with lidar_imu_calib result.
     xyz = extrinsic_T (metres)
     rpy = rotation matrix to Euler ZYX (radians) -->
<joint name="base_to_lidar" type="fixed">
  <parent link="base_link"/>
  <child link="lidar_link"/>
  <origin xyz="0.012 -0.003 0.131" rpy="0.0 0.0 0.0"/>  <!-- CALIB: example only -->
</joint>
```

NOTE: The URDF values and the FAST-LIO2 YAML values must be consistent — both encode the same lidar→IMU transform. The URDF is used by the TF tree (RViz visualization, Nav2 footprint), not by FAST-LIO2 directly.

3.4 Timestamp Offset (Optional)

scripts/calibrate_extrinsics.sh also outputs a time offset:

```
# Time offset between lidar and IMU timestamps (seconds)
# Write into fast_lio_hesai_qt64.yaml:
common:
  time_offset_lidar_to_imu: 0.0005   # <- from spin test; target < 0.001 s
```

4 Camera-Lidar Extrinsic → robot.urdf.xacro (base_to_camera)

The camera-lidar extrinsic locates the RealSense D435i in the robot frame. It is currently a rough tape-measure placeholder. It is used by Nav2 for obstacle detection (if depth is enabled) and by RViz for visualization. For SLAM the extrinsic is less critical than the lidar-IMU extrinsic.

4.1 Current Placeholder

robot.urdf.xacro line 124 (current placeholder):

```
<origin xyz="0.20 0.0 0.20" rpy="0.0 0.0 0.0"/>
# xyz: 20 cm forward, 0 lateral, 20 cm above base_link
# rpy: camera faces +X (forward) — approximate
```

4.2 Tape-Measure Method (Sufficient for Most Uses)

Measure from base_link origin (center of wheel axle, floor level):

```
# Required measurements (mm):
#  base_link_x → camera lens centre (along robot forward)
#  base_link_y → camera lens centre (left-right, 0 if centred)
#  base_link_z → camera lens centre (vertical height)
# Convert to metres and enter in robot.urdf.xacro base_to_camera joint:
<origin xyz="0.205 0.0 0.195" rpy="0.0 0.0 0.0"/>   # example
```

4.3 Full Extrinsic Calibration (Optional — Use if Depth Obstacles Required)

Run calibrate_cam_lidar.sh for precise extrinsic (checkerboard method):

```
# Requires checkerboard pattern and stationary lidar+camera running
bash scripts/calibrate_cam_lidar.sh

# Output: 4x4 transformation matrix T_cam_in_lidar
# Convert to base_link frame:
# T_cam_in_base = T_lidar_in_base * T_cam_in_lidar

# Write result into robot.urdf.xacro base_to_camera joint
```

NOTE: The RealSense D435i has factory-calibrated intrinsics stored in EEPROM. The camera intrinsic calibration (focal length, distortion) does not need to be repeated unless the camera was disassembled or physically damaged.

5 GPS Lever Arm → navsat_transform.yaml

The GPS lever arm is the offset from the robot base_link origin to the GPS antenna phase centre. It is needed by navsat_transform to correctly project GPS lat/lon into the robot-frame odometry.

5.1 Current URDF Value

robot.urdf.xacro base_to_gps joint (current):

```
<origin xyz="0.0 0.0 0.42" rpy="0.0 0.0 0.0"/>
# GPS antenna 42 cm above base_link — physical measurement
# Verify this matches the actual antenna mounting height
```

5.2 Measure the Lever Arm

- Measure in **all three axes** from the base_link origin (wheel axle centre, floor level) to the antenna phase centre.
- Phase centre is not the top of the antenna connector — it is typically 5–15 mm above the antenna body. Check the antenna datasheet.
- Enter the result in robot.urdf.xacro base_to_gps and rebuild.

5.3 Magnetic Declination

src/diy_localization/config/navsat_transform.yaml:

```
# CALIB: Look up the competition site magnetic declination from NOAA:
# https://ngdc.noaa.gov/geomag/calculators/magcalc.shtml
# Enter: competition venue lat/lon, current date; copy "Declination" in radians

magnetic_declination_radians: 0.0    # <- replace with NOAA value (example: -0.2153)
```

5.4 yaw_offset

Measure the GPS-to-robot heading offset:

```
# yaw_offset = angle from robot +X axis to magnetic North, measured clockwise
# Default: π/2 (robot +X faces East)
#
# Simple field measurement:
#  1. Place robot facing a known compass bearing (e.g. due East = 0 deg from North + 90 deg)
#  2. Read GPS heading from /gps/fix or /imu/data
#  3. yaw_offset = GPS heading - robot compass bearing (in radians)

yaw_offset: 1.5707963267948966       # <- replace with measured value
```

NOTE: A wrong yaw_offset causes navsat_transform to produce a GPS trajectory that is rotated relative to the lidar map. The error manifests as the GPS track curving away from the driven path in RViz. Adjust yaw_offset by the angular discrepancy observed.

6 EKF Process Noise Tuning → ekf_local.yaml

The EKF1 (odom frame) process noise covariance matrix Q encodes how much uncertainty the filter adds to its state estimate each time step. Too small = filter over-trusts predictions, ignores measurements. Too large = filter over-trusts measurements, becomes jumpy.

6.1 Current Values (Starting Point)

src/diy_localization/config/ekf_local.yaml — key diagonal values:

```
# Q matrix diagonal (position, orientation, linear velocity)
# x=0.05, y=0.05, z=0.06   — position uncertainty per cycle
# yaw=0.06                  — heading uncertainty per cycle
# vx=0.025, vy=0.025        — velocity uncertainty per cycle
```

6.2 Tuning Procedure (AprilTag Calibration)

The recommended method from diy-sad.html §6.3 uses AprilTag ground-truth poses recorded while FAST-LIO2 is running. Drive a known path past AprilTags, compare EKF output to tag ground truth, and adjust Q values to minimize RMS error.

- Mount an AprilTag at a known position in the test area.
- Drive the robot at 0.3 m/s past the tag at < 3 m range.
- Record a bag: /odometry/filtered, /camera/color/image_raw, /tf
- Post-process: compare EKF output to known tag position → compute residual.
- Increase diagonal Q value for noisy axes; decrease for under-correcting axes.
- Iterate until RMS error < 5 cm over a 10 m run.

6.3 Quick Sanity Check (No AprilTag)

Static test to check EKF covariance convergence:

```
# Start localization, keep robot stationary for 30 seconds
# Watch covariance in /odometry/filtered:
ros2 topic echo /odometry/filtered | grep -A 36 "pose.*covariance"

# Good result: x[0], y[7] diagonal entries converge to < 0.01 (static)
# If they grow over time: lidar odometry is drifting; check FAST-LIO2 params
```

7 Rebuild & Validation Checklist

After updating any calibration parameter, follow this checklist to verify the change is correctly integrated before the next mapping or competition session.

7.1 After Any YAML Change

- ■ Confirm YAML syntax is valid: python3 -c "import yaml; yaml.safe_load(open('config/fast_lio_hesai_qt64.yaml'))"
- ■ Rebuild diy_localization package: colcon build --packages-select diy_localization
- ■ Re-source ROS overlay: source install/setup.bash
- ■ Launch localization with use_rviz:=true and confirm no error messages at startup
- ■ Confirm FAST-LIO2 prints convergence message (not "IMU not ready")

7.2 After URDF Change

- ■ Rebuild diy_robot_description: colcon build --packages-select diy_robot_description
- ■ Confirm robot model in RViz has all links in correct position
- ■ Verify TF tree: ros2 run tf2_tools view_frames — all expected frames present
- ■ Check base_to_lidar transform: ros2 run tf2_ros tf2_echo base_link lidar_link
- ■ Check base_to_camera transform: ros2 run tf2_ros tf2_echo base_link camera_link

7.3 After Lidar-IMU Extrinsic Change

- ■ Run a short 2-minute mapping session with offline_mapping.launch.py
- ■ Verify map in RViz: walls should be straight verticals, no fan-shaped spread
- ■ Drive the figure-8 calibration path again and confirm map closes without gaps
- ■ Check that extrinsic_est_en: false in FAST-LIO2 config (disable online estimation)
- ■ Compare RMS residual with previous calibration — new value should be lower

7.4 After GPS Calibration Change

- ■ Drive robot in a straight line East for 20 m on open ground
- ■ In RViz: /gps/filtered points should overlap with /odometry/filtered path
- ■ Offset < 0.5 m for DGNSS, < 0.05 m for RTK fixed
- ■ Check navsat_transform output: /odometry/gps publishes at 10 Hz
- ■ Confirm EKF2 /odometry/global stays within 1 m of /odometry/filtered after 60 s

7.5 Final Pre-Competition CALIB Audit

Search for remaining placeholder markers in all config files:

```
# Check fast_lio config for placeholder values (0.1 is placeholder for acc/gyr_cov)
grep -n "CALIB" src/diy_localization/config/fast_lio_hesai_qt64.yaml
grep -n "CALIB" src/diy_localization/config/navsat_transform.yaml
grep -n "CALIB" src/diy_robot_description/urdf/robot.urdf.xacro

# Check for identity extrinsics (still placeholder):
grep -n "extrinsic_T: \[ 0.0, 0.0, 0.0" src/diy_localization/config/fast_lio_hesai_qt64.yaml

# Expected: zero remaining CALIB lines with 0.0 placeholder values
```

WARNING: If any grep above returns a line, that calibration step is incomplete. Do not proceed to competition with placeholder values.

End of Calibration Integration Guide. For the calibration scripts themselves see scripts/calibrate_*.sh. For detailed camera calibration procedures see the Camera Calibration Guide. For nav2 parameter tuning after localization is working see the Nav2 Tuning Guide.